lecture 4 quick sort

Probability Practice

6 sided die
Jack and Jill roll it

P(both get same sides) = $6(\frac{1}{6} \cdot \frac{1}{6}) = \frac{1}{6}$

Jack \ Jill	1	2	3	4	5	6
1	11					
2		22				
3			33			
4				44		
5					55	
6						66

total number of outcomes: 36
target outcomes: 6

$\frac{6}{36} = \frac{1}{6}$

also think about making strings with alphabet

Algorithm	Best Case	Worst Case	Avg Time	Space
Selection	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$
Insertion	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Merge	$O(n \log(n))$	$O(n \log(n))$	$O(n \log(n))$	$O(n)$

general fact: when best and worst case are the same, average time is the same

merge() uses $O(n)$ space, reuse the same space
since you are merging at most 1 at a time

Goal: same run time as merge sort but less space

Quick Sort

Intuition:
merge sort: had easy split and hard stitch
quick sort: have hard split and easy stitch
partition

Quick Sort Partition

Def: a list A is partitioned on element x IF:

$\leq x$	x	$\geq x$

example: partition on 15

9 18 20 15 7 → 9 7 15 20 18

→ 9 7 15 18 20

→ 7 9 15 18 20

→ 7 9 15 20 18

15 is at the sorted position

2 numbers on left and 2 numbers on right of 15
there are $2 \cdot 2 = 4$ ways to organize them

Partition Algorithm

With temp space
1. put less than pivot in left
2. put greater than pivot in right
3. copy first array, pivot value, and second array

Without temp space
1. first pivot and swap it with the first element so the pivot is the first element
2. look over the rest of the list with 2 pointers
3. increment left until you see an element greater than pivot
4. decrement right until you see an element less than pivot
5. If the pointers cross, exit the loop

You will be given the pivot index

---

pivot index
def(A, lo, hi, p): low index, high index ✱ recall var hi is N

```
swap A[lo] <-> A[p]        put pivot at beginning
v ∈ A[lo]                  pivot
i ∈ lo                     left pointer
j ∈ hi                     right pointer
while true:
    while A[++i] < v:      } left pointer →
        if i+1 ≥ hi:       }
            break
    while A[--j] > v:      } right pointer ←
        pass
    if i ≥ j:
        swap A[lo] <-> A[j]
        return j
    swap A[i] <-> A[j]
```

---

quick sort
- use partition to split the array
- then call quick sort on each partition

```
def quick_sort(A, lo, hi):

    if (hi - lo) ≤ 1: return A

    q = partition(A, lo, hi, ___)
    quick_sort(A, lo, q)
    quick_sort(A, q+1, hi)
    return A
```

≤ x	x	x ≥

---

picking the pivot

if pivot = lo and input is always sorted

Quick sort Recurrence Relation

recall $E[x] = \sum_x \frac{1}{n} \cdot p(x)$

recurrence: $C_n = (n+1) + \frac{1}{n}\sum_{p=0}^{n-1}[C_p + C_{n-p-1}]$

Theorem: Quicksort takes $O(n \log(n))$ expected time when all elements are distinct and the pivot is randomly chosen

output the same

$$C_n = (n+1) + \frac{1}{n}\sum_{p=0}^{n-1}[C_p + C_{n-p-1}]$$
$$C_n = (n+1) + \frac{2}{n}\sum_{p=0}^{n-1} C_p$$

there is no easy formula for this

usually, subtract $C_n - C_{n-1}$ to see if a lot of term cancel.

the $\frac{1}{n}$ is making the cancelation harder

$$nC_n - (n-1)C_{n-1} = n\left[(n+1) + \frac{2}{n}\sum_{p=0}^{n-1} C_p\right] - (n-1)\left[(n-1+1) + \frac{2}{n-1}\sum_{p=0}^{n-1-1} C_p\right]$$
$$nC_n - (n-1)C_{n-1} = n(n+1) + 2\sum_{p=0}^{n-1}(C_p) - n(n-1) - 2\sum_{p=0}^{n-2}(C_p)$$
$$nC_n - (n+1)C_{n-1} = n^2 + n - n^2 + n + 2C_{n-1}$$
$$nC_n - (n-1)C_{n-1} = 2n + 2C_{n-1}$$
$$nC_n = 2n + 2C_{n-1} + nC_{n-1} - C_{n-1}$$
$$nC_n = 2n + C_{n-1} + nC_{n-1}$$

$$C_n = 2 + \frac{n+1}{n} C_{n-1}$$

$$\frac{C_n}{n+1} = \frac{2}{n+1} + \frac{C_{n-1}}{n}$$

let $S_n = \frac{C_n}{n+1} \rightarrow S_{n-1} = \frac{C_{n-1}}{n-1+1} = \frac{C_{n-1}}{n}$

$S_n = \frac{2}{n+1} + S_{n-1}$ for $n \geq 3$ $S_0 = 0, S_1 = 0, S_2 = 1$

$$S_n = \frac{C_n}{n+1} = \left[\frac{2}{n+1} + \frac{2}{n} + \frac{2}{n-1} + \dots + \frac{2}{4}\right] + 1 + 0 + 0$$
$S_3 = \frac{2}{4}$ S_2 S_1 S_0

$$C_n = 2(n+1)\left(\frac{1}{n+1} + \frac{1}{n} + \frac{1}{n-1} + \dots \frac{1}{4}\right) + 1(n+1)$$
$$C_n = 2(n+1)(H_{n+1} - H_2) + (n+1)$$

$H_n = \Theta(\log(n))$

$C_n = \Theta(n \log(n))$